

PYTHON STANDART KÜTÜPHANESİ

Gerçek dünya kullanım senaryolarında bu kütüphane olmazsa olmazdır. Standart kütüphanedeki (**standart library**) çok kullanılan modüller, `datetime`, `os`, `sys`, `json` ve `math` modülleridir.

Zaman ve Tarih Yönetimi

Mühendislik uygulamalarında, verinin "ne" olduğu kadar "ne zaman" üretildiği de kritiktir. Bir sensör verisinin **zaman damgası** (**timestamp**), bir işlemin süresi veya bir sistemin log kayıtları için zaman yönetimi hataya yer bırakmamalıdır. Python, bu karmaşıklığı yönetmek için `datetime` modülünü sunar.

Zamanın Temsili ve Temel Sınıflar

`datetime` modülü, zamanı farklı katmanlarda ele alan dört ana sınıftan oluşur:

1. `date`: Sadece yıl, ay ve gün bilgilerini tutar (Takvim verisi).
2. `time`: Sadece saat, dakika, saniye ve mikro saniye bilgilerini tutar (Saat verisi).
3. `datetime`: Hem tarih hem de saat bilgilerini bir arada tutar.
4. `timedelta`: İki zaman noktası arasındaki farkı (süreyi) temsil eder.

Sistem Zamanına Erişim

Bir mühendis için en temel işlem, sistemin mevcut zamanını okumaktır.

```
from datetime import datetime
simdi = datetime.now()
print(f"Sistem Zamanı: {simdi}")
print(f"Yıl: {simdi.year}, Ay: {simdi.month}, Gün: {simdi.day}")
```

Zaman Aritmetiği

Zaman üzerinde toplama ve çıkarma işlemleri yapmak için `timedelta` sınıfı kullanılır. Bu sınıf, özellikle periyodik işlemler (örneğin; "bundan 30 gün sonra ne olacak?") için kullanılır.

```
from datetime import datetime, timedelta
bugun = datetime.now()
bir_hafta_sonra = bugun + timedelta(days=7)
print(f"Bir hafta sonraki tarih: {bir_hafta_sonra.date()}")
```

Zaman aritmetiğinde doğrudan tam sayı toplamak yerine `timedelta` kullanmak, artık yıllar (**leap years**) ve ayların gün sayıları gibi karmaşık takvim hesaplamalarını Python'un otomatik yapmasını sağlar.

Biçimlendirme

Zaman verisi genellikle dış dünyadan metin (string) olarak gelir veya dış dünyaya metin olarak sunulur. İki fonksiyon biçimlendirme için kullanılır;

- `strftime` (**string format time**): Zaman objesini metne dönüştürür.
- `strptime` (**string parse time**): Metin halindeki zamanı objeye dönüştürür.

```
# Obje -> Metin
print(bugun.strftime("%d/%m/%Y %H:%M:%S")) # 16/02/2026 14:30:00

# Metin -> Obje
tarih_metni = "2026-05-19"
tarih_objesi = datetime.strptime(tarih_metni, "%Y-%m-%d")
```

Zaman Dilimleri

Mühendislikte en çok yapılan hatalardan biri, **zaman dilimini** (**timezone**) göz ardı etmektir.

- Saf (**naive**): Zaman dilimi bilgisi içermeyen, sadece yerel saati gösteren objelerdir.
- Farkında (**aware**): UTC (Evrensel Zaman) referans alınarak zaman dilimi bilgisi eklenmiş objelerdir.

Küresel sistemlerde her zaman UTC kullanılması bir mühendislik standardıdır. Örnek olarak bir algoritmanın çalışma hızını ölçmek için **datetime** farklarından yararlanabiliriz:

```
baslangic = datetime.now()

# ... Algoritma veya İşlem ...
toplam = sum(range(1000000))

bitis = datetime.now()
gecen_sure = bitis - baslangic
print(f"İşlem süresi: {gecen_sure.total_seconds()} saniye.")
```

Metin İşleme ve Düzenli İfadeler

Yazılım dünyasında **ham metin verisi** (**raw text**) genellikle düzensizdir. Bir mühendis için binlerce satırlık bir hata **iz kaydından** (**log**) sadece IP adreslerini ayıklamak veya bir veri setindeki tarih formatlarını standart hale getirmek manuel olarak imkansızdır. **Düzenli ifadeler** (**regex**), metinler içerisinde belirli desenleri tanımlamak ve bu desenlere uyan verileri bulmak, değiştirmek veya doğrulamak için kullanılan özelleşmiş bir dildir.

Python'da bu işlemler için standart kütüphanede yer alan **re** modülü kullanılır.

Temel Karakterler ve Belirleyiciler

Regex, kendine has bir sembol dili kullanır. **Belirleyiciler** (**meta character**) aşağıda verilmiştir;

Desen (pattern)	Açıklama
.	Herhangi bir karakteri temsil eder.
^	Satır başı karakterini temsil eder.
\$	Satır sonu karakterini temsil eder.
*	Bir karakterin sıfır veya daha fazla tekrarını temsil eder.
+	Bir karakterin bir veya daha fazla tekrarını temsil eder.
?	Bir karakterin sıfır veya bir kez tekrarını temsil eder.
{n}	Bir karakterin tam n kez tekrarını temsil eder.
[abc]	a, b veya c karakterlerinden birini temsil eder.
\d	Herhangi bir rakamı (0-9) temsil eder.
\w	Herhangi bir alfanümerik karakteri (harf, rakam, alt çizgi) temsil eder.
\s	Boşluk karakteri temsil eder.

re Modülü Temel Fonksiyonları

Python'da en sık kullanılan Regex yöntemleri şunlardır:

- **re.search()**: Metin içinde deseni arar ve ilk bulduğu konumu (**match object**) döndürür.
- **re.findall()**: Desene uyan tüm kısımları bir liste olarak döndürür.
- **re.sub()**: Desene uyan kısımları başka bir metinle değiştirir.
- **re.compile()**: Bir deseni önceden derleyerek (pre-compile) performans artışı sağlar.

Gruplandırma ve Veri Ayıklama

Parantez **()** kullanarak desenin belirli kısımlarını gruplandırabilir ve sadece o gruplara erişebiliriz.

E-posta doğrulama ve ayıklama örneği verilmiştir;

```
import re

metin = "İletişim için ilhan.ozkan@kurum.com veya test_kullanici@gmail.com kullanın."
# E-posta deseni: [karakterler]@[alan_adi].[uzanti]
desen = r"([\w.-]+)@([\w.-]+)"
eslesmeler = re.findall(desen, metin)

for kullanıcı, alan in eslesmeler:
    print(f"Kullanıcı: {kullanıcı}, Alan Adı: {alan}")
```

Açgözlü Eşleşme

Regex varsayılan olarak açgözlüdür (**greedy**), yani bulabildiği en uzun eşleşmeyi alır. Mühendislik uygulamalarında (örneğin HTML etiketlerini ayıklarken) bazen en kısa eşleşmeye ihtiyaç duyarız. Bunun için `?` operatörü kullanılır.

Regex ifadeleri güçlüdür ancak karmaşıklıkla okunabilirliği ve bakım maliyeti (**maintenance cost**) artar. Bir mühendis olarak, çok karmaşık bir regex yazmak yerine, işlemi mantıksal adımlara bölmek veya açıklayıcı yorumlar eklemek (**verbose modu**) kodun kalitesini artıracaktır.

Derlenmiş Desenler ve Performans

Eğer aynı desen binlerce satırlık bir veri üzerinde döngü içinde kullanılacaksa, deseni bir kez derlemek işlemci zamanından tasarruf sağlar.

```
# Deseni bir kez derle
ip_deseni = re.compile(r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}")

# Döngü içinde hızlıca kullan
with open("access.log", "r") as f:
    for satir in f:
        if ip_deseni.search(satir):
            # ... işlem ...
```

Dosya Yönetimi ve Bağlam Yöneticileri

Bir yazılımın dış dünya ile veri alışverişi yapmasının en temel yolu dosya sistemidir. Mühendislik uygulamalarında log kayıtları, konfigürasyon dosyaları veya sensör verilerinin kalıcı hale getirilmesi için dosya işlemleri hayati önem taşır. Ancak dosyalar, işletim sistemi tarafından sınırlı sayıda sunulan **kaynaklardır** (**resource**). Açılan her dosyanın işi bittiğinde kapatılması, dosya ile işlemlerin en kısa sürede bitirilmesi, sistem kaynaklarının tükenmemesi için bir zorunluluktur.

Dosya Açma ve Erişim Modları

Python'da bir dosyaya erişmek için `open()` fonksiyonu kullanılır. Bu fonksiyon, işletim sistemi seviyesindeki **dosya tanıtıcısı** (**file descriptor**) nesnesine bir referans oluşturur.

```
# Temel kullanım (Önerilmeyen yöntem)
dosya = open("veriler.txt", "w")
dosya.write("Mühendislik Verisi")
dosya.close() # Kapatma unutulursa kaynak sızıntısı oluşur!
```

Erişim Modları:

1. **'r'**: Okuma (read) modu. Dosya yoksa istisna oluşur.
2. **'w'**: Yazma (write) modu. Dosya varsa içeriği siler, yoksa oluşturur.
3. **'a'**: Ekleme (append) modu. Dosyanın sonuna veri ekler.
4. **'b'**: Verilerin bilgisayar belleğindeki gibi tutulduğu ikili (binary) mod. Resim, ses veya derlenmiş dosya işlemleri için kullanılır.
5. **'r+'**: Hem okuma ve hem de yazma Modu.
6. **'rb'**: İkili dosyayı okuma Modu.

7. **'wb'**: İkili dosyayı yazma Modu.

Bağlam Yöneticileri

Mühendislikte "hata toleransı" esastır. Eğer dosya açıkken bir hata oluşursa, klasik yöntemle `close()` komutu hiçbir zaman çalışmayabilir. Python bu riski ortadan kaldırmak için bağlam yöneticisi (**context manager**) yapısını sunar. Bu yapı sayesinde dosya `with` bloğu kullanıldığında, işlem bittikten sonra veya bir hata oluşsa dahi dosya otomatik olarak kapatılır.

```
with open("deney_sonuc_lari.csv", "r") as dosya:
    icerik = dosya.read()
    # İşlem bittiğinde sistem dosyayı otomatik kapatır.
```

`with` kullanımı, C++ dilindeki Kaynak Edinimi Başlatma İşlemidir (**Resource Acquisition Is Initialization-RAII**) prensibine benzer. Kaynak kullanımı, kodun kapsamı (scope) ile sınırlıdır.

Dosya Okuma Teknikleri

Büyük veri setleriyle (**big data**) çalışırken tüm dosyayı bir kerede belleğe (RAM) yüklemek sistemi kilitleyebilir. Mühendisçe yaklaşım, veriyi satırlar halinde işlemektir. Genelde gözle okunabilir metin dosyalarını işlerken kullanılır.

```
with open("buyuk_veri_seti.txt", "r") as dosya:
    for satir in dosya:
        # Her adımda sadece bir satır bellekte tutulur
        print(satir.strip())
```

Yapılandırılmış Veri Saklama

Standart metin dosyaları yerine, verileri **yapılandırılmış** (**structured**) bir formatta saklamak veri bütünlüğü için önemlidir. Verileri gözle okunabilir şekilde barındıran ve en çok kullanılan formatlar **XML** (**Extensible Markup Language**) ve **JSON** (**Java Script Object Notation**) formatıdır. Python dilinin `json` modülü ile nesneleri doğrudan dosyaya kaydedebiliriz.

```
import json
veriler = {"sensor_id": 101, "sicaklik": 24.5}
with open("config.json", "w") as f:
    json.dump(veriler, f) # Veriyi JSON formatında dosyaya yazar
```

Dosya ve Dizin Kontrolleri

Bir dosyayı açmadan önce varlığını kontrol etmek, programın çökmesini engeller. Bu durumu `os` nesnesi ile kontrol edebiliriz.

```
import os

if os.path.exists("rapor.txt"):
    print("Dosya zaten mevcut.")
else:
    print("Yeni dosya oluşturuluyor...")
```

Sistem Parametreleri ve Etkileşimi

Python'un `sys` modülü, **yorumlayıcının** (**interpreter**) **çalışma anındaki** (**runtime**) ortamına ve sistem kaynaklarına doğrudan erişim sağlar. `os` modülü işletim sistemiyle ilgilenirken, `sys` modülü daha çok Python'un kendisi ve bilgisayar donanımı arasındaki düşük seviyeli parametrelerle ilgilenir. Bir mühendis için bu modül, bellek yönetimi ve sistem entegrasyonu açısından vazgeçilmezdir.

`sys` modülü, Python'un **abstraction** (**soyutlama**) seviyesinden bir adım aşağı inerek donanıma yaklaşmamızı sağlar. Özellikle bellek optimizasyonu ve terminal tabanlı araçlar geliştirirken bu modülün sunduğu parametreler yazılımın performansını doğrudan etkiler.

Komut Satırı Argümanları

Yazılan bir **betiğin** (**script**), terminal üzerinden dışarıdan parametreler alarak çalışması gerekebilir. `sys.argv`, komut satırından girilen argümanları bir liste olarak saklar.

1. `sys.argv[0]`: Çalıştırılan dosyanın adıdır.
2. `sys.argv[1:]`: Dışarıdan girilen parametrelerdir.

```
import sys
# terminal: python hesapla.py 10 20
print(f"Çalışan dosya: {sys.argv[0]}")
if len(sys.argv) > 2:
    sayi1 = int(sys.argv[1])
    sayi2 = int(sys.argv[2])
    print(f"Toplam: {sayi1 + sayi2}")
```

Bellek Yönetimi

Bir mühendis için yazdığı kodun ne kadar bellek (RAM) tükettiği hayati önem taşır. `sys.getsizeof()` fonksiyonu, bir Python nesnesinin bellekte kapladığı alanı **bayt** (**byte**) cinsinden döndürür.

```
import sys

liste = [i for i in range(1000)]
sozluk = {i: i for i in range(1000)}

print(f"Listenin bellek boyutu: {sys.getsizeof(liste)} byte")
print(f"Sözlüğün bellek boyutu: {sys.getsizeof(sozluk)} byte")
```

Standart Giriş, Çıkış ve Hata Akışları

Bilgisayar mimarisinde I/O (Girdi/Çıktı) işlemleri üç ana kanal üzerinden yürür: `stdin` (standart giriş), `stdout` (standart çıkış) ve `stderr` (standart hata).

- `sys.stdout`: `print()` fonksiyonunun arka planda kullandığı kanaldır.
- `sys.stderr`: Hata mesajlarını sisteme iletmek için kullanılır. Mühendislik uygulamalarında hataları normal çıktılardan ayırmak için bu kanalın kullanılması standart bir pratiktir.

Dosya İşlemlerinde Özel Hata Türleri

En çok olan hata türleri aşağıda verilmiştir;

```
try:
    with open('dosya.txt', 'r') as f:
        icerik = f.read()
except FileNotFoundError:
    print("Dosya bulunamadı!")
except PermissionError:
    print("Dosya erişim izni yok!")
except IOError as e:
    print(f"Dosya hatası: {e}")
```

Dosyaların Ön Tanımlı Olarak Aranacağı Dizinler

Python bir modülü **içe aktarırken** (**import**), belirli dizinleri kontrol eder. `sys.path`, bu dizinlerin listesini tutar. Eğer kendi yazdığınız kütüphaneleri Python'un bulmasını istiyorsanız, bu listeye müdahale edebilirsiniz.

```
import sys
# Python'un kütüphane arama yollarına yeni bir dizin eklemek
sys.path.append("C:/KendiKutuphanelerim")
```

Programdan Çıkış

Bir hata durumunda veya görev tamamlandığında programın akışını güvenli bir şekilde sonlandırmak ve işletim sistemine bir **çıkış kodu** (**exit code**) döndürmek için kullanılır.

- `sys.exit(0)`: Başarılı çıkış.
- `sys.exit(1)`: Hatalı çıkış (Sistem mühendisliğinde hata takip mekanizmaları için kritiktir).

Yorumlayıcı ve Platform Bilgileri

Yazılımın hangi platformda (Windows, Linux, macOS) ve hangi Python sürümünde çalıştığını denetlemek, kodun **taşınabilirliği** (**portability**) için önemlidir.

```
print(f"İşletim Sistemi Platformu: {sys.platform}")
print(f"Python Sürümü: {sys.version}")
```

Matematiksel Hesaplamalar

Bilgisayar mimarisinde matematiksel işlemler işlemcinin **aritmetik işlem birimi** (**arithmetic logic unit-ALU**) birimi tarafından yürütülür. Python'un `math` modülü, C diliyle yazılmış olan standart matematik kütüphanesine doğrudan erişim sağlayarak, karmaşık hesaplamaların hem hızlı hem de standartlara uygun (IEEE 754) yapılmasını sağlar¹⁶.

Mühendislik hesaplamalarında **kayan noktalı sayıların** (**floating point**) işlemci seviyesindeki davranışı bazen istenmeyen yuvarlama hatalarına neden olabilir.

- `math` Modülü: C dilinin standart matematik kütüphanesi üzerine inşa edilmiştir. Trigonometri, logaritma ve sabitler (π , e) gibi donanım destekli hızlı hesaplama araçlarını sunar.
- `decimal` Modülü: Finansal veya yüksek hassasiyetli mühendislik hesaplamaları için float veri tipinin zafiyetlerini giderir. $0.1 + 0.2$ 'nin tam olarak 0.3 etmesini sağlayan deterministik bir hesaplama yapısı sunar.

Matematiksel Sabitler

Mühendislik formüllerinde kullanılan evrensel sabitler, modül içerisinde yüksek hassasiyetle tanımlanmıştır.

- `math.pi`: π sayısı (3.141592...).
- `math.e`: Euler sayısı (2.718281...).
- `math.tau`: π değerini temsil eden sabit.

Sayı Yuvarlama ve Sınır Değerler

Sayısal analizlerde, kayan noktalı bir sayının (float) tam sayıya nasıl dönüştürüleceği işlemin sonucunu etkiler.

- `math.ceil(x)`: Sayıyı yukarıya, en yakın tam sayıya yuvarlar.
- `math.floor(x)`: Sayıyı aşağıya, en yakın tam sayıya yuvarlar.
- `math.trunc(x)`: Sayının ondalık kısmını atarak sadece tam sayı kısmını bırakır.

Üs, Logaritmik ve Güç Fonksiyonları

Donanım tasarımı veya veri analitiği gibi alanlarda kullanılan temel fonksiyonlardır.

- `math.pow(x, y)`: x sayısının y inci kuvvetini alır.
- `math.sqrt(x)`: Sayının karekökünü hesaplar.
- `math.log(x, base)`: Belirtilen tabanda logaritma hesaplar.

¹⁶ <https://docs.python.org/3/library/math.html>

Trigonometrik İşlemler

Trigonometrik fonksiyonları **Radyan** cinsinden hesaplar. Mühendislikte sıkça kullanılan derece-radyan dönüşümleri için `math` modülü yardımcı fonksiyonlar sunar.

- `math.sin()`, `math.cos()`, `math.tan()`: Temel trigonometrik fonksiyonlar.
- `math.radians(x)`: Dereceyi radyana çevirir.
- `math.degrees(x)`: Radyanı dereceye çevirir.

```
import math

derece = 90
radyan = math.radians(derece)

print(f"90 derecenin sinüsü: {math.sin(radyan)}") # Çıktı: 1.0
```

Özel Durumlar

Bilgisayar bilimlerinde bazı matematiksel sonuçlar standart sayılarla ifade edilemez.

- `math.inf`: **Sonsuz** (**infinity**) değerini temsil eder.
- `math.nan`: **Sayı olmayan** (**not a number-NaN**) değerini temsil eder. Hatalı veri girişlerini kontrol etmek için kritiktir.

`math.isclose()` fonksiyonu, iki kayan noktalı sayının birbirine "yeterince yakın" olup olmadığını kontrol etmek için kullanılır. `if a == b` kontrolü float sayılarda yuvarlama hataları nedeniyle risklidir; bu yüzden toleranslı bir karşılaştırma her zaman daha güvenlidir.

Pisagor teoremini kullanarak bir dik üçgenin hipotenüsünü hesaplayan program örneği aşağıda verilmiştir;

```
import math

def hipotenus_hesapla(a, b):
    # math.hypot doğrudan karelerin toplamının kökünü alır
    return math.hypot(a, b)

kenar1 = 3
kenar2 = 4
print(f"Hipotenüs: {hipotenus_hesapla(kenar1, kenar2)}") # Çıktı: 5.0
```

Bilimsel Hesaplama

NumPy (**Numerical Python**), Python'da bilimsel hesaplama için temel kütüphanedir. Çok boyutlu **diziler** (**array**) ve bu diziler üzerinde yapılan matematiksel işlemler için optimize edilmiş fonksiyonlar sağlar.

NumPy, veri bilimi, makine öğrenmesi ve bilimsel hesaplama projelerinin temelini oluşturur. Python'un standart listelerine göre çok daha hızlı ve bellek verimlidir. Bu nedenle çok önemlidir.

`pip install numpy` ile kurulumu yapılır. Kullanımı ise `numpy` dosya.py şeklindedir.

```
import time

# Python listesi ile işlem
python_liste = list(range(1000000))
baslangic = time.time()
python_sonuc = [x * 2 for x in python_liste]
python_sure = time.time() - baslangic

# NumPy array ile işlem
numpy_array = np.arange(1000000)
baslangic = time.time()
```

```
numpy_sonuc = numpy_array * 2
numpy_sure = time.time() - baslangic

print(f"Python listesi: {python_sure:.4f} saniye")
# Çıktı: Python listesi: 0.0523 saniye
print(f"NumPy array: {numpy_sure:.6f} saniye")
# Çıktı: NumPy array: 0.001234 saniye
print(f"Hız artışı: {python_sure/numpy_sure:.1f}x")
# Çıktı: Hız artışı: 42.4x

# NumPy çok daha hızlıdır!
```

NumPy Dizileri

NumPy'nin temel veri yapısı ndarray (**n-dimensional array**) yani çok boyutlu dizidir.

```
# Python listesinden dizi oluşturma
liste = [1, 2, 3, 4, 5]
arr = np.array(liste)
print(arr)
# Çıktı: [1 2 3 4 5]
print(type(arr))
# Çıktı: <class 'numpy.ndarray'>

# İki boyutlu dizi (matris)
matris = np.array([[1, 2, 3], [4, 5, 6]])
print(matris)
# Çıktı: [[1 2 3]
# Çıktı:  [4 5 6]]

# Üç boyutlu dizi
ucboyutlu = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(ucboyutlu.shape)
# Çıktı: (2, 2, 2)
```

Özel Dizi Oluşturma Fonksiyonları

```
# Sıfırlarla dolu array
sifirlar = np.zeros((3, 4))
print(sifirlar)
# Çıktı: [[0. 0. 0. 0.]
# Çıktı:  [0. 0. 0. 0.]
# Çıktı:  [0. 0. 0. 0.]]

# Birlerle dolu array
birler = np.ones((2, 3))
print(birler)
# Çıktı: [[1. 1. 1.]
# Çıktı:  [1. 1. 1.]]

# Belirli bir değerle dolu array
dolu = np.full((2, 2), 7)
print(dolu)
# Çıktı: [[7 7]
# Çıktı:  [7 7]]

# Birim matris (Identity matrix)
birim = np.eye(3)
print(birim)
# Çıktı: [[1. 0. 0.]
# Çıktı:  [0. 1. 0.]
# Çıktı:  [0. 0. 1.]]
```



```
# Belirli aralıkta sayılar
aralik = np.arange(0, 10, 2)
print(aralik)
# Çıktı: [0 2 4 6 8]

# Eşit aralıklı sayılar
esit_aralik = np.linspace(0, 1, 5)
print(esit_aralik)
# Çıktı: [0.  0.25 0.5  0.75 1.  ]

# Rastgele sayılar
rastgele = np.random.rand(3, 3) # 0-1 arası
print(rastgele)
# Çıktı: [[0.5488 0.7152 0.6028]
# Çıktı: [0.5449 0.4237 0.6459]
# Çıktı: [0.4376 0.8918 0.9637]]

# Normal dağılım
normal = np.random.randn(3, 3) # Ortalama 0, standart sapma 1
print(normal)
# Çıktı: [[-0.2342  0.3452 -0.9423]
# Çıktı: [ 1.1234 -0.4563  0.2345]
# Çıktı: [ 0.5678 -0.1234  0.8901]]
```

Dizi Özellikleri

```
# Boyut (dimensions)
print(f"Boyut (ndim): {arr.ndim}")
# Çıktı: Boyut (ndim): 2

# Şekil (shape) - her boyuttaki eleman sayısı
print(f"Şekil (shape): {arr.shape}")
# Çıktı: Şekil (shape): (3, 4)

# Toplam eleman sayısı
print(f"Boyut (size): {arr.size}")
# Çıktı: Boyut (size): 12

# Veri tipi
print(f"Veri tipi (dtype): {arr.dtype}")
# Çıktı: Veri tipi (dtype): int64

# Bellekte kapladığı alan (bytes)
print(f"Her elemanın boyutu: {arr.itemsize} bytes")
# Çıktı: Her elemanın boyutu: 8 bytes
print(f"Toplam bellek: {arr.nbytes} bytes")
# Çıktı: Toplam bellek: 96 bytes
```

Dizi İndisleri ve Dilimleme

```
# Tek eleman erişimi
print(arr[0])
# Çıktı: 10
print(arr[-1])
# Çıktı: 50

# Dilimleme (slicing)
print(arr[1:4])
# Çıktı: [20 30 40]
print(arr[:,2])
# Çıktı: [10 30 50]
```

```
# İki boyutlu array
matris = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Tek eleman erişimi
print(matris[0, 0])
# Çıktı: 1
print(matris[1, 2])
# Çıktı: 6

# Satır erişimi
print(matris[1])
# Çıktı: [4 5 6]

# Sütun erişimi
print(matris[:, 1])
# Çıktı: [2 5 8]

# Alt matris
print(matris[0:2, 1:3])
# Çıktı: [[2 3]
# Çıktı: [5 6]]

# Boolean indeksleme
arr = np.array([1, 2, 3, 4, 5])
maske = arr > 3
print(arr[maske])
# Çıktı: [4 5]

# Koşullu indeksleme
print(arr[arr % 2 == 0])
# Çıktı: [2 4]
Array Şekillendirme
print(arr)
# Çıktı: [ 0  1  2  3  4  5  6  7  8  9 10 11]

# Yeniden şekillendirme (reshape)
matris = arr.reshape(3, 4)
print(matris)
# Çıktı: [[ 0  1  2  3]
# Çıktı: [ 4  5  6  7]
# Çıktı: [ 8  9 10 11]]

# Otomatik boyut hesaplama (-1 kullanarak)
matris2 = arr.reshape(4, -1) # 4 satır, otomatik sütun
print(matris2)
# Çıktı: [[ 0  1  2]
# Çıktı: [ 3  4  5]
# Çıktı: [ 6  7  8]
# Çıktı: [ 9 10 11]]

# Düzleştirme (flatten)
duz = matris.flatten()
print(duz)
# Çıktı: [ 0  1  2  3  4  5  6  7  8  9 10 11]

# Transpozisyon (satır ve sütunları yer değiştirme)
transpoz = matris.T
print(transpoz)
# Çıktı: [[ 0  4  8]
# Çıktı: [ 1  5  9]
# Çıktı: [ 2  6 10]
```

```
# Çıktı: [ 3  7 11]
```

Matematiksel İşlemler

Temel Aritmetik

```
b = np.array([10, 20, 30, 40])
```

```
# Toplama
```

```
print(a + b)
```

```
# Çıktı: [11 22 33 44]
```

```
# Çıkarma
```

```
print(b - a)
```

```
# Çıktı: [ 9 18 27 36]
```

```
# Çarpma (element-wise)
```

```
print(a * b)
```

```
# Çıktı: [ 10  40  90 160]
```

```
# Bölme
```

```
print(b / a)
```

```
# Çıktı: [10. 10. 10. 10.]
```

```
# Üs alma
```

```
print(a ** 2)
```

```
# Çıktı: [ 1  4  9 16]
```

```
# Skaler işlemler
```

```
print(a + 10)
```

```
# Çıktı: [11 12 13 14]
```

```
print(a * 2)
```

```
# Çıktı: [2 4 6 8]
```

İstatistiksel Fonksiyonlar

```
# Toplam
```

```
print(f"Toplam: {np.sum(arr)}")
```

```
# Çıktı: Toplam: 45
```

```
print(f"Satır toplamaları: {np.sum(arr, axis=1)}")
```

```
# Çıktı: Satır toplamaları: [ 6 15 24]
```

```
print(f"Sütun toplamaları: {np.sum(arr, axis=0)}")
```

```
# Çıktı: Sütun toplamaları: [12 15 18]
```

```
# Ortalama
```

```
print(f"Ortalama: {np.mean(arr)}")
```

```
# Çıktı: Ortalama: 5.0
```

```
# Standart sapma
```

```
print(f"Standart sapma: {np.std(arr):.2f}")
```

```
# Çıktı: Standart sapma: 2.58
```

```
# Minimum ve maksimum
```

```
print(f"Minimum: {np.min(arr)}")
```

```
# Çıktı: Minimum: 1
```

```
print(f"Maksimum: {np.max(arr)}")
```

```
# Çıktı: Maksimum: 9
```

```
# Minimum ve maksimumun indeksi
```

```
print(f"Min indeks: {np.argmin(arr)}")
```

```
# Çıktı: Min indeks: 0
```

```
print(f"Max indeks: {np.argmax(arr)}")
```

```
# Çıktı: Max indeks: 8
```

```
# Medyan
print(f"Medyan: {np.median(arr)}")
# Çıktı: Medyan: 5.0

# Yüzdelik dilimler (percentiles)
print(f"25. yüzdelik: {np.percentile(arr, 25)}")
# Çıktı: 25. yüzdelik: 3.0
print(f"75. yüzdelik: {np.percentile(arr, 75)}")
# Çıktı: 75. yüzdelik: 7.0
```

Matematiksel Fonksiyonlar

```
# Trigonometrik fonksiyonlar
print(f"Sinüs: {np.sin(arr)}")
# Çıktı: Sinüs: [0.  0.5  0.71 0.87 1.  ]
print(f"Kosinüs: {np.cos(arr)}")
# Çıktı: Kosinüs: [1.  0.87 0.71 0.5  0.  ]

# Üstel ve logaritma
arr2 = np.array([1, 2, 3, 4, 5])
print(f"e^x: {np.exp(arr2)}")
# Çıktı: e^x: [ 2.72  7.39 20.09 54.60 148.41]
print(f"log(x): {np.log(arr2)}")
# Çıktı: log(x): [0.  0.69 1.10 1.39 1.61]
print(f"log10(x): {np.log10(arr2)}")
# Çıktı: log10(x): [0.  0.30 0.48 0.60 0.70]

# Karekök
print(f"√x: {np.sqrt(arr2)}")
# Çıktı: √x: [1.  1.41 1.73 2.  2.24]

# Mutlak değer
arr3 = np.array([-1, -2, 3, -4, 5])
print(f"|x|: {np.abs(arr3)}")
# Çıktı: |x|: [1 2 3 4 5]

# Yuvarlama
arr4 = np.array([1.234, 2.567, 3.891])
print(f"Round: {np.round(arr4, 2)}")
# Çıktı: Round: [1.23 2.57 3.89]
print(f"Floor: {np.floor(arr4)}")
# Çıktı: Floor: [1. 2. 3.]
print(f"Ceil: {np.ceil(arr4)}")
# Çıktı: Ceil: [2. 3. 4.]

Lineer Cebir İşlemleri
# Matris çarpımı
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Element-wise çarpım
print(f"Element-wise çarpım:\n{A * B}")
# Çıktı: Element-wise çarpım:
# Çıktı: [[ 5 12]
# Çıktı:  [21 32]]

# Matris çarpımı (dot product)
print(f"Matris çarpımı:\n{np.dot(A, B)}")
# Çıktı: Matris çarpımı:
# Çıktı: [[19 22]
# Çıktı:  [43 50]]

# @ operatörü ile matris çarpımı
print(f"A @ B:\n{A @ B}")
```

```
# Çıktı: A @ B:
# Çıktı: [[19 22]
# Çıktı:  [43 50]]

# Determinant
det = np.linalg.det(A)
print(f"Determinant: {det}")
# Çıktı: Determinant: -2.0

# Ters matris (inverse)
A_inv = np.linalg.inv(A)
print(f"Ters matris:\n{A_inv}")
# Çıktı: Ters matris:
# Çıktı: [[-2.   1. ]
# Çıktı:  [ 1.5 -0.5]]

# Özdeğerler ve özvektörler
ozdegerler, ozvektorler = np.linalg.eig(A)
print(f"Özdeğerler: {ozdegerler}")
# Çıktı: Özdeğerler: [-0.37  5.37]
print(f"Özvektörler:\n{ozvektorler}")
# Çıktı: Özvektörler:
# Çıktı: [[-0.82 -0.42]
# Çıktı:  [ 0.57 -0.91]]
```

Yayın

NumPy modülünde, farklı şekillerdeki dizileri otomatik olarak uyumlu hale getirmek için **yayın** (**broadcasting**) yapılır.

```
# Skaler ile arrayb
arr = np.array([1, 2, 3])
print(arr + 10)
# Çıktı: [11 12 13]

# Farklı şekillerde array'ler
a = np.array([[1, 2, 3], [4, 5, 6]]) # (2, 3)
b = np.array([10, 20, 30])          # (3,)

print(a + b)
# Çıktı: [[11 22 33]
# Çıktı:  [14 25 36]]

# Sütun vektörü ile matris
a = np.array([[1, 2, 3], [4, 5, 6]]) # (2, 3)
b = np.array([[10], [20]])           # (2, 1)

print(a + b)
# Çıktı: [[11 12 13]
# Çıktı:  [24 25 26]]
Array Birleştirme ve Ayırma
# Array birleştirme
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Dikey birleştirme (vstack)
dikey = np.vstack((a, b))
print(dikey)
# Çıktı: [[1 2 3]
# Çıktı:  [4 5 6]]

# Yatay birleştirme (hstack)
```

```

yatay = np.hstack((a, b))
print(yatay)
# Çıktı: [1 2 3 4 5 6]

# Concatenate (axis belirtilerek)
birlesmis = np.concatenate((a, b))
print(birlesmis)
# Çıktı: [1 2 3 4 5 6]

# Array ayırma
arr = np.array([1, 2, 3, 4, 5, 6])

# Eşit parçalara ayırma
parcalar = np.split(arr, 3)
print(parcalar)
# Çıktı: [array([1, 2]), array([3, 4]), array([5, 6])]

# Belirli indekslerde ayırma
parcalar2 = np.split(arr, [2, 4])
print(parcalar2)
# Çıktı: [array([1, 2]), array([3, 4]), array([5, 6])]

Kopyalama ve View
# View (görünüm) - aynı veriyi gösterir
arr = np.array([1, 2, 3, 4, 5])
view = arr.view()
view[0] = 999
print(arr)
# Çıktı: [999  2  3  4  5] # Orijinal array değişti!

# Copy (kopya) - bağımsız yeni array
arr = np.array([1, 2, 3, 4, 5])
kopya = arr.copy()
kopya[0] = 999
print(arr)
# Çıktı: [1 2 3 4 5] # Orijinal array değişmedi
print(kopya)
# Çıktı: [999  2  3  4  5]

```

Pratik Örnekler

Veri Normalizasyonu Örneği

```

# Min-Max normalizasyon (0-1 arası)
veri = np.array([10, 20, 30, 40, 50])

min_val = np.min(veri)
max_val = np.max(veri)
normalize_edilmis = (veri - min_val) / (max_val - min_val)

print(normalize_edilmis)
# Çıktı: [0.  0.25 0.5  0.75 1.  ]

# Z-score normalizasyon (standartlaştırma)
ortalama = np.mean(veri)
std = np.std(veri)
standartlastirilmis = (veri - ortalama) / std

print(standartlastirilmis)
# Çıktı: [-1.41 -0.71  0.   0.71  1.41]

```

Mesafe Hesaplama Örneği

```

# İki nokta arasındaki Öklid mesafesi
nokta1 = np.array([1, 2, 3])

```

```
nokta2 = np.array([4, 5, 6])

# mesafe = np.sqrt(np.sum((nokta1 - nokta2) ** 2))
print(f"Öklid mesafesi: {mesafe:.2f}")
# Çıktı: Öklid mesafesi: 5.20

# Alternatif: linalg.norm kullanarak
mesafe2 = np.linalg.norm(nokta1 - nokta2)
print(f"Mesafe (linalg.norm): {mesafe2:.2f}")
# Çıktı: Mesafe (linalg.norm): 5.20
```

Görüntü İşleme

```
# Gri tonlamalı görüntü (8-bit, 0-255 arası)
goruntu = np.random.randint(0, 256, (5, 5), dtype=np.uint8)
print("Orijinal görüntü:")
print(goruntu)

# Parlaklık artırma
parlak_goruntu = np.clip(goruntu + 50, 0, 255).astype(np.uint8)
print("\nParlak görüntü:")
print(parlak_goruntu)

# Kontrast artırma
kontrast_goruntu = np.clip(goruntu * 1.5, 0, 255).astype(np.uint8)
print("\nYüksek kontrastlı görüntü:")
print(kontrast_goruntu)

# Negatif görüntü
negatif_goruntu = 255 - goruntu
print("\nNegatif görüntü:")
print(negatif_goruntu)
```

Polinomial Regresyon

```
# Veri noktaları
x = np.array([1, 2, 3, 4, 5])
y = np.array([2.1, 3.9, 6.2, 8.1, 9.8])

# 2. derece polinom fit etme
katsayilar = np.polyfit(x, y, 2)
print(f"Katsayılar (a, b, c): {katsayilar}")
# Çıktı: Katsayılar (a, b, c): [-0.04  2.09  0.04]

# Tahmin yapma
x_yeni = np.linspace(1, 5, 50)
y_tahmin = np.polyval(katsayilar, x_yeni)

# Hata hesaplama
y_fit = np.polyval(katsayilar, x)
mse = np.mean((y - y_fit) ** 2)
print(f"Ortalama Kare Hata (MSE): {mse:.4f}")
# Çıktı: Ortalama Kare Hata (MSE): 0.0123
```

Performans İpuçları

```
# 1. Vektörleştirme (Vectorization) kullanın
# Kötü: Döngü ile
arr = np.arange(1000000)
sonuc = np.zeros(1000000)
for i in range(len(arr)):
    sonuc[i] = arr[i] * 2
```

```
# İyi: Vektörleştirilmiş
arr = np.arange(1000000)
sonuc = arr * 2

# 2. In-place işlemler kullanın (bellek tasarrufu)
arr = np.arange(1000000)
arr *= 2 # arr = arr * 2 yerine

# 3. Uygun veri tipi seçin
float64 yerine float32 kullanmak bellek kullanımını yarıya indirir
arr_buyuk = np.zeros(1000000, dtype=np.float64) # 8 MB
arr_kucuk = np.zeros(1000000, dtype=np.float32) # 4 MB

# 4. Broadcasting kullanın
# Kötü
a = np.arange(1000).reshape(10, 100)
b = np.arange(100)
sonuc = np.zeros_like(a)
for i in range(10):
    sonuc[i] = a[i] + b

# İyi
sonuc = a + b # Broadcasting otomatik
```

En İyi NumPy Uygulamaları

```
# 1. Doğru veri tipini kullan
entero = np.array([1, 2, 3], dtype=np.int32) # Tam sayılar için
ondalik = np.array([1.0, 2.0], dtype=np.float32) # Ondalıklı için
karmasik = np.array([1+2j], dtype=np.complex64) # Karmaşık sayılar için

# 2. Shape kontrolü yapın
def guvenli_toplama(a, b):
    """Shape uyumluluğunu kontrol eden toplama"""
    if a.shape != b.shape:
        raise ValueError(f"Shape uyumsuzluğu: {a.shape} vs {b.shape}")
    return a + b

# 3. Bellek yönetimi
# Büyük array'leri del ile silin
buyuk_array = np.zeros((10000, 10000))
# İşlemler...
del buyuk_array # Belleği serbest bırak

# 4. Kod okunabilirliği için isim verin
sutun_ortalamalari = np.mean(matris, axis=0) # axis=0 yerine açıklayıcı
satir_ortalamalari = np.mean(matris, axis=1) # axis=1 yerine açıklayıcı
```

Bilimsel Hesaplama Özeti

Veri bilimi ve analizi, Makine öğrenmesi, Görüntü işleme, Bilimsel hesaplama, Finans ve ekonomi ile Mühendislik simülasyonları alanlarında çok kullanılan NumPy'nin temel özellikleri:

- Hız: Python listelerinden 10-100x daha hızlı
- Çok Boyutlu Diziler: `ndarray` veri yapısı
- Matematiksel İşlemler: Vektörleştirilmiş işlemler
- Yayın: Farklı boyutlardaki dizileri otomatik uyumlu hale getirme
- Lineer Cebir: Matris işlemleri, özdeğerler, determinant
- İstatistik: Ortalama, standart sapma, medyan, vb.
- Rastgele Sayılar: Çeşitli dağılımlardan rastgele sayı üretimi

Olasılık ve İstatistiksel Analiz Araçları

Mühendislikte sistem davranışlarını simüle etmek veya toplanan verileri analiz etmek için matematiksel modellerden yararlanırız. Python, bu ihtiyaçlar için `random` ve `statistics` modüllerini sunar.

Rastlantısallık ve Simülasyon

Bilgisayar bilimlerinde gerçek rastlantısallık zordur; bu yüzden **sözde rastlantısal** (**pseudo-random**) sayılar kullanılır. `random` modülü, bu sayıları üretmek için deterministik algoritmalar kullanır.

- `random.random()`: 0,0 ile 1,0 arasında rastgele bir float sayı üretir.
- `random.randint(a, b)`: Belirtilen aralıkta (a ve b dahil) bir tam sayı üretir.
- `random.choice(liste)`: Bir liste içinden rastgele bir eleman seçer.
- `random.seed(x)`: Rastlantısal sayı üreticini sabitler. **Simülasyonların tekrarlanabilirliği** (**reproducibility**) için mühendislikte hayati önem taşır.

Temel Veri Analizi

Deneyisel verilerin merkezi eğilimlerini ve yayılımını ölçmek için kullanılır.

- `statistics.mean()`: Verilerin aritmetik ortalamasını hesaplar.
- `statistics.median()`: Verilerin orta değerini bulur (aykırı değerlerin etkisini azaltmak için kullanılır).
- `statistics.stdev()`: Standart sapmayı hesaplar. Verinin ne kadar güvenilir veya değişken olduğunu ölçen temel mühendislik metriğidir.

Yüksek Performanslı Döngü Yönetimi

Büyük veri setleri üzerinde iterasyon yaparken **bellek verimliliği** (**memory efficiency**) en öncelikli konudur. `itertools` modülü, karmaşık döngüleri ve kombinasyonel işlemleri bellek dostu **yineleyiciler** (**iterator**) ile yönetmemizi sağlar.

Sonsuz ve Koşullu Yineleyiciler

- `itertools.count(start, step)`: Belirli bir sayıdan başlayarak sonsuza kadar artan bir sayı dizisi üretir. Sayaç mekanizmaları için idealdir.
- `itertools.cycle(iterable)`: Bir veri grubunu sonsuz bir döngüye sokar.

Kombinasyonel Araçlar

Karmaşık olasılık hesaplamalarında ve sistem konfigürasyonlarını test ederken kullanılır:

- `itertools.product()`: Kartezyen çarpım oluşturur.
- `itertools.permutations()`: Elemanların tüm sıralı dizilişlerini hesaplar.
- `itertools.combinations()`: Elemanların sırasız gruplarını (kombinasyonlarını) oluşturur.

`itertools` fonksiyonları veriyi belleğe bir anda yüklemeyi, **ihtiyaç duyuldukça üretir** (**lazy evaluation**). Bu sayede milyonlarca kombinasyon içeren işlemler bile sistemi kilitlemeden yürütülebilir.

İleri Seviye Veri Yapıları

Standart liste ve sözlüklerin performans olarak yetersiz kaldığı algoritmalarda, özelleşmiş koleksiyonlar (**collection**) kullanılır.

`namedtuple`: Veriyi hem indis hem de anahtar kelime ile erişim sağlayan, bellek dostu **değiştirilemez** (**immutable**) bir yapıdır.

`deque` (**double-ended queue**): Algoritma bölümlerinde işlediğimiz "Kuyruk" yapısının, her iki uçtan da O(1) karmaşıklığıyla veri ekleme-çıkarma yapabilen optimize edilmiş halidir.

JSON Veri Değişim Formatı

Günümüzde yazılımlar tek başlarına değil, bir ağ (network) üzerinden diğer sistemlerle konuşarak çalışırlar. Farklı dillerle (Python, C++, C#) yazılmış sistemlerin birbirini anlayabilmesi için ortak bir veri diline ihtiyaç vardır. **JSON** (JavaScript Object Notation), bu ihtiyacı karşılayan, dilden bağımsız, metin tabanlı ve insan tarafından okunabilir en popüler veri değişim formatıdır.

JSON Yapısı ve Python Nesneleriyle İlişkisi

JSON yapısı, bir mühendis için yabancı değildir; çünkü Python'daki **sözlük** (dict) ve **liste** (list) yapılarının birebir metin karşılığıdır.

- Nesneler: Süslü parantez { } içinde tutulan **anahtar-değer** (key-value) çiftleridir. (Python'daki dict)
- Diziler: Köşeli parantez [] içinde tutulan sıralı veri gruplarıdır. (Python'daki list)

Serileştirme

Python nesnelerini (bellekteki canlı verileri) bir dosyaya yazılacak veya ağdan gönderilecek bir metin formatına dönüştürme işlemine **serileştirme** (serialization) denir. Bunun tersi işleme ise **deserileştirme** (deserialization) adı verilir.

- `json.dumps()`: Python nesnesini bir JSON karakter dizisine (string) dönüştürür.
- `json.loads()`: JSON formatındaki bir metni Python nesnesine dönüştürür.

Dosya Tabanlı İşlemler

Eğer veri bir string değişkeni yerine doğrudan bir dosyaya yazılacaksa, "s" takısı olmayan fonksiyonlar kullanılır.

JSON ile Veri Bütünlüğü

JSON kullanmak, verinin tip güvenliğini sağlar. Örneğin, Python'daki bir **None** değeri JSON'da **null** olarak, **True** değeri ise **true** olarak saklanır. Bu sayede veri, başka bir dile (örneğin C#'a) aktarıldığında o dilin standartlarına göre doğru şekilde yorumlanır.

JSON verisi metin tabanlı olduğu için **utf-8** karakter kodlamasıyla saklanır. Büyük veri setlerinde JSON dosyaları çok fazla yer kaplayabilir; bu durumlarda veriyi sıkıştırmak veya **pickle** gibi **ikili** (binary) formatları kullanmak performans açısından değerlendirilmelidir.

Örnek Uygulama

Aşağıda sistem bilgisi ve veri saklama konulu örnek verilmiştir

```
import os
import sys
import json

def sistem_ozeti_hazirla():
    # Sistemden bilgi toplama
    bilgi = {
        "isletim_sistemi": os.name,
        "python_surumu": sys.version.split()[0],
        "mevcut_dizin": os.getcwd()
    }

    # Veriyi JSON formatına dönüştürme (Serileştirme)
    json_cikti = json.dumps(bilgi, indent=4)
    print("Sistem Analiz Raporu:")
    print(json_cikti)

sistem_ozeti_hazirla()
```

Bu programın çıktısı aşağıdaki gibi olacaktır;

```
Sistem Analiz Raporu:  
{  
    "isletim_sistemi": "nt",  
    "python_surumu": "3.13.4",  
    "mevcut_dizin": "D:\\Python\\PSamples"  
}
```